1. What is self in Python?

The **self parameter** represents the **current instance (object)** of the class.

When you create an object from a class,
self refers to *that particular object* inside the class methods.

💡 self helps Python know **which object's data** you're working with.

🧱 **2. Syntax: How It Looks**

```
class Example:
    def show(self):
        print("This is a method inside a class")
```

When you create an object:

```
obj = Example()
obj.show()
```

Internally, Python does this:

```
Example.show(obj)
```

That means:

✅ The object obj is automatically passed as the first argument (self) to the method.

🧠 **3. So Why Do We Need self?**

Without self, your class methods wouldn't know which object's attributes to use or modify!

It helps:

- Bind **data** (attributes) with **functions** (methods)
- Differentiate between **instance variables** and **local variables**

🚗 **4. Example: Understanding self Clearly**

```
class Car:
```

```
    def __init__(self, brand, color):
        self.brand = brand     # instance variable
        self.color = color

    def show_details(self):
        print(f"Car Brand: {self.brand}, Color: {self.color}")

# Creating objects
car1 = Car("Toyota", "Red")
car2 = Car("Tesla", "Black")

car1.show_details()
car2.show_details()
```

📄 **Output:**

```
Car Brand: Toyota, Color: Red
Car Brand: Tesla, Color: Black
```

✅ Here:

- self.brand refers to **car1.brand** when you call car1.show_details()
- self.brand refers to **car2.brand** when you call car2.show_details()

So, self keeps track of **which object** is calling the method.

🧠 **5. Without self (Common Mistake)**

Let's see what happens if you remove it 👇

```
class Car:
    def __init__(brand, color):   # ❌ Missing self
        brand = brand
```

```
    color = color
```

This will raise an error:

TypeError: __init__() takes 2 positional arguments but 3 were given

Why?
Because when you do car1 = Car("Toyota", "Red"), Python automatically sends the instance as the first argument — and there's no parameter to receive it.

💡 6. Why Is It Called self?

It's **not a keyword** — just a **naming convention**.

You can name it anything (e.g., this, obj, me), but **self is the standard** —
and every Python developer expects it.

✅ Best Practice: Always use self for readability and consistency.

🧩 7. Difference Between self and cls

Keyword	Used In	Refers To	Commonly Used With
self	Instance methods	Object	def method(self):
cls	Class methods	Class itself	@classmethod def method(cls):

Example 👇

```
class Example:
    class_variable = "Shared"

    def show_self(self):
        print("Called by object:", self)

    @classmethod
    def show_cls(cls):
```

```
    print("Called by class:", cls)


obj = Example()
obj.show_self()    # instance method
Example.show_cls() # class method
```

⚙️ 8. Real-Life Analogy

Think of a **class as a blueprint** and **objects as houses**.

- The blueprint (class) defines properties like color, number of doors.
- Each house (object) has its own color and doors.
- self helps identify **which specific house** you're referring to.

```
class House:
    def __init__(self, color):
        self.color = color

    def paint(self, new_color):
        print(f"Painting {self.color} house to {new_color}")
        self.color = new_color

h1 = House("Blue")
h2 = House("Green")

h1.paint("Red")    # affects only h1
h2.paint("Yellow") # affects only h2
```

📜 Output:

```
Painting Blue house to Red
```

Painting Green house to Yellow

✅ self ensures each object maintains its **own data**.

🧪 9. Practice Exercise

Try this yourself 👇

```
# Exercise: Understanding self

# 1. Create a class 'Laptop' with:

#    - attributes: brand, ram, storage

# 2. Add a method 'show_specs()' that displays all the details.

# 3. Create two Laptop objects with different specs and call the method.

# 4. Observe how 'self' keeps each object’s data separate.
```

📜 10. Summary Table

Concept	Meaning	Example
self	Refers to current object	self.brand = brand
Used in	Instance methods	def display(self):
Automatically passed	Yes, when method called via object	obj.method()
Must be first parameter	Always	def __init__(self, ...)
Not a keyword	Just a convention	You can rename, but avoid it